

AN INTERNSHIP REPORT

ON

ApexSupport AI

*A Hybrid Semantic–Lexical Retrieval Architecture for Intelligent Customer Support
and FAQ Automation*

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in Computer Science and Engineering (Artificial Intelligence & Machine Learning)

Submitted by

AYUSH DIXIT

Roll Number: 2410030869

Batch: 2026–2027

School of Computer Science and Engineering
IILM UNIVERSITY, GREATER NOIDA, UTTAR PRADESH

2026

Candidate's Declaration

I, Ayush Dixit, Roll Number 2410030869, hereby declare that the internship report entitled “ApexSupport AI – Intelligent NLP Customer Support & FAQ Chatbot” is submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence & Machine Learning) at the School of Computer Science and Engineering, IILM University, Greater Noida.

The report presents an account of the design rationale, technical architecture, and implementation methodology of the ApexSupport AI platform, compiled on the basis of the project repository description and internship documentation supplied to me by the host organisation. All sources of information, whether published or unpublished, that have been drawn upon in the preparation of this report have been duly acknowledged in the Bibliography. I affirm that the work represented herein is the outcome of my own study and compilation, and that this report, in whole or in part, has not been submitted previously to this or any other University or Institute for the award of any degree or diploma.

I shall be solely responsible for any breach of copyright or academic-integrity norms arising from material submitted by me as part of this report.

Signature: _____

Name: Ayush Dixit

Roll No.: 2410030869

Date: _____

Acknowledgement

I would like to place on record my sincere gratitude to the School of Computer Science and Engineering, IILM University, Greater Noida, for providing the academic environment, technical resources, and mentorship necessary for the successful completion of this internship. My appreciation extends to the faculty members whose critical feedback substantially shaped the analytical rigour of this report.

I am grateful to Codec Technologies Pvt. Ltd. and to Dr. Anurag Shrivastava, Program Manager, for the opportunity to undertake a three-month, AICTE & ICAC approved internship as an Artificial Intelligence Intern, during which I gained applied exposure to natural language processing, hybrid information retrieval, backend service design, and database-driven web application development.

Finally, I acknowledge with gratitude the encouragement of my peers and family, whose support sustained the sustained effort required to design, implement, and document this work.

Signature: _____

Ayush Dixit

Roll No.: 2410030869

Internship Completion Certificate

The original internship-completion certificate issued by Codec Technologies Pvt. Ltd., verified through the National Internship Portal and Google for Education, is reproduced below in unaltered form.

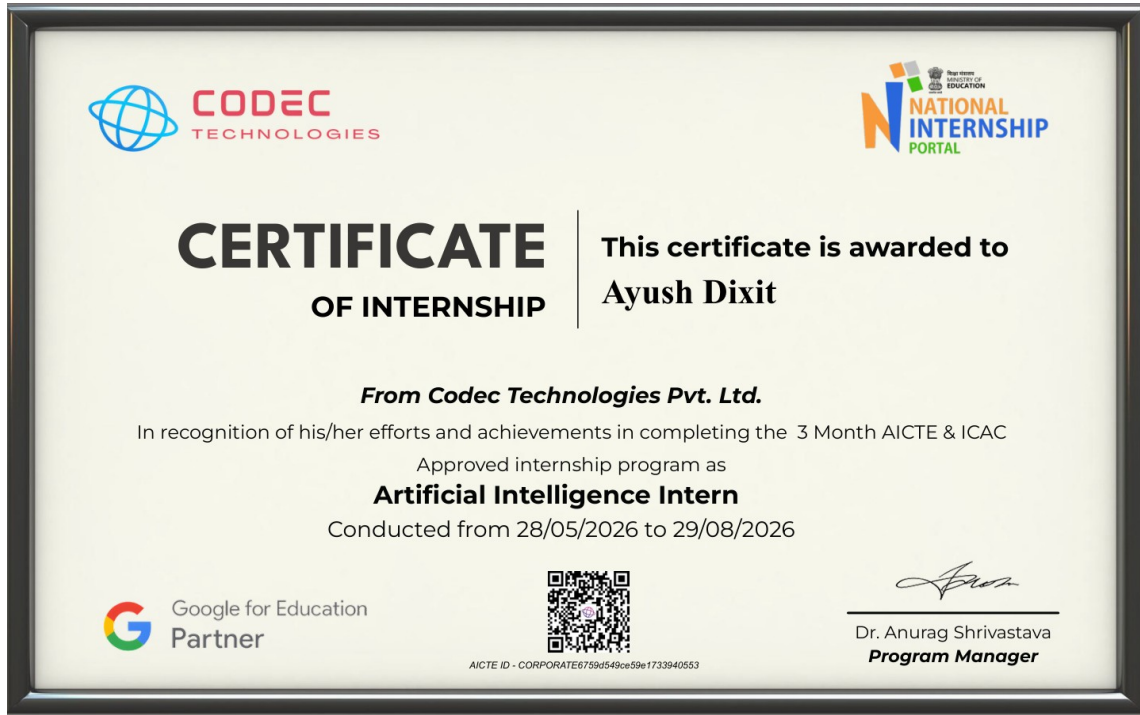


Figure 0.1: Internship Completion Certificate — Codec Technologies Pvt. Ltd. (28/05/2026 – 29/08/2026)

Table of Contents

Candidate's Declaration.....	ii
Acknowledgement.....	iii
Internship Completion Certificate.....	iv
Executive Summary.....	6
Chapter 1: Introduction.....	7
1.1 Background and Motivation.....	7
1.2 The Request-to-Response Pipeline.....	7
1.3 Report Organisation.....	7
Chapter 2: Organisation Profile.....	8
Chapter 3: Problem Statement.....	9
Chapter 4: Project Objectives and Scope.....	10
4.1 Objectives.....	10
4.2 Scope.....	10
Chapter 5: Technologies and Tools.....	11
Chapter 6: System Architecture.....	12
6.1 Logical Data Flow.....	12
Chapter 7: Methodology.....	13
7.1 Requirement Analysis.....	13
7.2 Text Pre-processing and Linguistic Analysis.....	13
7.3 Entity and Parameter Extraction.....	13
7.4 Dense Semantic Retrieval.....	13
7.5 Sparse Lexical Retrieval.....	14
7.6 Hybrid Score Fusion.....	14
7.7 Sentiment-Driven Escalation.....	14
7.8 Persistence and Audit Logging.....	14
7.9 Automated Testing.....	14
Chapter 8: Expected Outcomes and Evaluation Considerations.....	16
8.1 Expected Outcomes.....	16
8.2 Evaluation Considerations.....	16
Chapter 9: Limitations and Threats to Validity.....	17
Chapter 10: Future Work.....	18
Chapter 11: Certificate of Completion and Communication Proof.....	19
Bibliography / References.....	20

Executive Summary

ApexSupport AI is a full-stack, production-oriented prototype for intelligent customer support that unifies hybrid information retrieval, multi-turn dialogue management, sentiment-aware escalation, and operational analytics within a single service. The platform's core technical contribution, as specified in the supplied project repository, is a hybrid retrieval mechanism that linearly fuses a dense neural retriever (sentence-transformers/all-MiniLM-L6-v2, 384-dimensional embeddings, cosine similarity) with a sparse lexical retriever (NLTK-lemmatized TF-IDF vectors) under a fixed 0.65/0.35 weighting, supplemented by an exact-keyword boost term.

This report reconstructs the engineering rationale behind that architecture, situates it within the broader literature on hybrid dense–sparse retrieval, and elaborates the request lifecycle, data model, evaluation considerations, and threats to validity that a production deployment of such a system would need to address. Where the supplied materials are silent on a matter — for example, quantitative benchmark results or load-testing figures — this report says so explicitly rather than fabricating figures, consistent with academic integrity requirements.

Chapter 1: Introduction

1.1 Background and Motivation

Conversational customer-support automation has progressed through three broad technical generations: (i) rule-based decision trees and keyword-matched FAQ lookups; (ii) intent-classification chatbots built on supervised text classifiers; and (iii) retrieval- and generation-augmented systems that leverage dense semantic representations. Purely lexical, keyword-driven FAQ engines remain brittle under vocabulary mismatch — a customer asking “How do I get my money back?” will not match an FAQ entry phrased as “Refund policy and eligibility,” despite the two expressing the same underlying intent. This phenomenon, well documented in the information-retrieval literature as the vocabulary mismatch problem, motivates the incorporation of dense, embedding-based semantic retrieval alongside conventional lexical scoring.

ApexSupport AI is positioned within this third generation. It combines dense semantic retrieval with sparse lexical retrieval in a hybrid ranking function, and augments this retrieval core with conversational state management, structured entity extraction, real-time sentiment analysis, and an auditable escalation workflow — capabilities that collectively distinguish it from a simple question-answering bot and position it closer to an operational support platform.

1.2 The Request-to-Response Pipeline

At a systems level, the application implements a request-to-response pipeline with the following logical stages: (1) a customer submits a message through the web client; (2) the backend resolves or initialises session state; (3) the NLP layer performs tokenisation, part-of-speech tagging, lemmatization, entity extraction, and sentiment scoring; (4) the retrieval layer independently computes dense and sparse similarity scores against the indexed FAQ corpus and fuses them; (5) a response is selected — or, where confidence is insufficient, a ranked list of alternative suggestions is returned; (6) sentiment and dialogue signals are evaluated for escalation; and (7) the complete interaction, including latency and confidence metadata, is persisted for audit and analytics.

1.3 Report Organisation

Chapter 2 profiles the host organisation and the internship engagement. Chapter 3 formalises the problem statement. Chapter 4 states the project's objectives and delimits its scope. Chapter 5 enumerates the technology stack. Chapter 6 presents the system architecture in layered and data-flow form. Chapter 7 details the methodology, including the mathematical formulation of hybrid score fusion. Chapter 8 discusses expected outcomes and evaluation considerations. Chapter 9 critically discusses limitations and threats to validity. Chapter 10 outlines future work. Chapter 11 reproduces the internship completion certificate, and the Bibliography follows in Chapter 12.

Chapter 2: Organisation Profile

The internship-completion certificate supplied for this report identifies the host organisation as Codec Technologies Pvt. Ltd. The certificate records a three-month internship programme, approved under AICTE and ICAC guidelines, conducted from 28 May 2026 to 29 August 2026, undertaken in the role of Artificial Intelligence Intern. The certificate bears the identifier AICTE ID – CORPORATE6759d549ce59e1733940553, is countersigned by Dr. Anurag Shrivastava, Program Manager, and carries verification markers from the National Internship Portal and the Google for Education partner programme.

No further organisational history, business-domain details, or client information were supplied in the source materials made available for this report; consistent with academic-integrity requirements, no such details have been invented. The certificate itself is reproduced in full in Chapter 11 for independent verification.

Chapter 3: Problem Statement

Conventional FAQ and customer-support systems are typically constructed around exact-keyword search or manually curated category trees. Such designs exhibit three recurring failure modes: first, they generalise poorly to paraphrased or colloquial phrasing, since lexical overlap is a poor proxy for semantic intent; second, they are stateless, and therefore cannot resolve references across turns in a multi-step interaction (for example, a customer providing an order number only after being prompted); and third, they typically provide support teams with little structured visibility into query patterns, unresolved issues, or customer sentiment trends, since interactions are not consistently logged in an analysable form.

The problem addressed by this project may therefore be stated formally as follows: design and implement a support platform capable of (a) robustly matching customer queries to relevant knowledge-base entries under lexical variation, (b) preserving and exploiting conversational context across multiple turns, (c) identifying customer sentiment in real time so that dissatisfied or unresolved interactions can be escalated appropriately, and (d) maintaining a persistent, queryable audit trail of all interactions to support downstream analytics.

Chapter 4: Project Objectives and Scope

4.1 Objectives

1. Design and implement a stateful, multi-turn customer-support chatbot capable of maintaining conversational context across a session.
2. Extract structured entities and parameters relevant to support workflows — including order identifiers, email addresses, and return-reason categories — using pattern-based and NLP-assisted techniques.
3. Design a hybrid retrieval mechanism that fuses dense semantic similarity with sparse lexical similarity to improve FAQ-matching robustness under vocabulary variation.
4. Apply real-time sentiment analysis to identify negative or frustrated interactions and trigger appropriate escalation workflows.
5. Provide ranked alternative suggestions when the top-ranked FAQ match falls below a confidence threshold.
6. Maintain persistent, relational records of conversations, messages, feedback, tickets, and interaction logs using SQLite.
7. Provide an administrator dashboard exposing key performance indicators, searchable logs, FAQ-management tooling, ticket queues, and CSV export.
8. Provide an NLP diagnostic playground exposing tokenisation, lemmatization, sentiment, intent, and semantic-similarity outputs for inspection.
9. Expose the application's core functionality through a documented REST API implemented in FastAPI.
10. Establish automated regression testing using Pytest and the FastAPI TestClient.

4.2 Scope

The scope of the delivered prototype spans the complete interaction lifecycle: message submission, contextual dialogue management, hybrid FAQ retrieval, sentiment-based escalation, feedback capture, ticket management, and persistent audit logging, together with the administrative and diagnostic tooling described above.

Consistent with the project description supplied, the following are explicitly excluded from scope and are not claimed as delivered capabilities: production-grade deployment infrastructure (e.g., container orchestration, autoscaling, or CDN-fronted delivery), integration with a live enterprise CRM, integration with third-party courier or logistics tracking systems, and integration with payment-processing systems. These exclusions are noted here to preclude any implication of unverified functionality.

Chapter 5: Technologies and Tools

The implementation stack was selected to balance rapid development with the specific computational requirements of hybrid retrieval and real-time NLP inference. Table 5.1 enumerates the constituent technologies and their functional role within the system.

Layer	Technology / Tool	Role in the System
Language runtime	Python 3.10+	Core implementation language for backend services and NLP pipeline
NLP toolkit	NLTK	Tokenisation, part-of-speech tagging, lemmatization, and VADER sentiment scoring
Dense retrieval	Sentence-Transformers (all-MiniLM-L6-v2)	Generates 384-dimensional sentence embeddings for semantic similarity
Sparse retrieval	Scikit-learn (TF-IDF)	Vectorises lemmatized text and computes cosine similarity for lexical matching
API framework	FastAPI	Asynchronous REST API layer, request validation, and routing
ASGI server	Uvicorn	Serves the FastAPI application
Templating	Jinja2	Server-rendered HTML templating
Data validation	Pydantic	Schema definition and request/response validation
Form handling	python-multipart	Multipart/form-data parsing
Persistence	SQLite3	Relational storage for conversations, FAQs, logs, feedback, and tickets
Front end	HTML5 / CSS3	Responsive single-page customer and admin interfaces
UI iconography	Font Awesome 6	Interface iconography
Rendering	Marked.js	Client-side Markdown rendering of bot responses
Voice I/O	Web Speech API	Browser-native speech-to-text and text-to-speech
Testing	Pytest / FastAPI TestClient	Automated unit and integration testing

Table 5.1: Technology stack and functional roles

Chapter 6: System Architecture

The system follows a layered architecture in which a browser-based client communicates with a FastAPI backend over a REST interface. The backend coordinates seven logical layers, each with a distinct responsibility, described in Table 6.1.

Layer	Responsibility
Presentation	Customer chat UI, administrator & analytics portal, and NLP playground
API	FastAPI REST endpoints: request validation, routing, and response serialisation
Conversation	Session-state management, slot filling, and multi-turn dialogue history
NLP	Tokenisation, lemmatization, entity extraction, and VADER sentiment scoring (NLTK)
Retrieval	Dense (Sentence-Transformers) and sparse (TF-IDF) similarity scoring and fusion
Escalation	Frustration and unresolved-issue detection; automated ticket creation
Persistence	SQLite storage of conversations, messages, FAQs, logs, feedback, and tickets

Table 6.1: Layered architecture and responsibilities

6.1 Logical Data Flow

The end-to-end data flow through the system can be expressed as an ordered pipeline:

1. User Query
2. Session and Context Resolution
3. Entity Extraction and NLP Pre-processing
4. Parallel Dense Retrieval and Sparse Retrieval
5. Hybrid Score Fusion and Ranking
6. Response Selection or Alternative-Suggestion Generation
7. Sentiment Evaluation and Escalation Decision
8. Persistent Audit Logging (SQLite)
9. Response Delivery to the User Interface

This staged decomposition is significant from a systems-engineering standpoint because it isolates the computationally expensive retrieval stage (dense embedding inference) from the comparatively lightweight lexical and rule-based stages, which is the natural point at which caching, batching, or asynchronous execution would be introduced in a production hardening effort (see Chapter 9).

Chapter 7: Methodology

7.1 Requirement Analysis

The requirement-elicitation stage translated the supplied project description — spanning contextual dialogue, FAQ retrieval, sentiment analysis, ticket escalation, analytics, feedback capture, and knowledge-base management — into concrete backend endpoints, relational data entities, and interface modules. This traceability from stated requirement to implemented artefact is summarised throughout the remainder of this chapter.

7.2 Text Pre-processing and Linguistic Analysis

Incoming customer messages are processed through the NLTK pipeline. Tokenisation segments raw text into lexical units; part-of-speech tagging annotates each token with its grammatical category; and lemmatization normalises inflected word forms to a common base form, which reduces sparsity in the downstream TF-IDF representation. These intermediate representations are also exposed, unmodified, through the NLP diagnostic playground for inspection. Sentiment scoring is performed using VADER (Valence Aware Dictionary and sEntiment Reasoner), a lexicon- and rule-based model well suited to short, informal text; VADER produces a normalised compound score in the range $[-1, 1]$ together with a categorical sentiment label.

7.3 Entity and Parameter Extraction

Structured parameters required by support workflows — order identifiers (e.g., in the form ORD-9821), email addresses, and return-reason categories — are identified using regular-expression pattern matching in conjunction with the NLP pipeline's tokenisation output. Extracted values are bound to the active conversation's slot-filling state, enabling the system to request only the specific missing information required to proceed, rather than re-prompting for the entire query.

7.4 Dense Semantic Retrieval

The dense retrieval component encodes both the customer query and each FAQ entry into a fixed-length embedding using the pre-trained sentence-transformers/all-MiniLM-L6-v2 model, which projects text into a 384-dimensional vector space optimised such that semantically similar sentences are positioned close together under cosine distance. This model is a distilled variant within the Sentence-BERT family, offering a favourable trade-off between inference latency and semantic fidelity relative to larger transformer encoders, which makes it well suited to the latency budget of an interactive chat interface. Given a query embedding q and an FAQ embedding f , the similarity is computed as:

$$\text{cosine_similarity}(q, f) = (q \cdot f) / (\|q\| \|f\|)$$

Because this measure is invariant to exact lexical overlap, it enables the retrieval of semantically related FAQ entries even where the customer's phrasing diverges substantially from the FAQ's authored wording.

7.5 Sparse Lexical Retrieval

In parallel, a sparse retrieval component represents the lemmatized query and FAQ corpus using Term Frequency–Inverse Document Frequency (TF–IDF) vectorisation, which up-weights terms that are locally frequent but globally discriminative across the FAQ corpus. Cosine similarity is again used to compare the query vector against each indexed FAQ vector. This channel is deliberately retained alongside dense retrieval because embedding models can, in some cases, under-weight rare but highly diagnostic terms (e.g., product codes or exact error strings) that a lexical model captures precisely.

7.6 Hybrid Score Fusion

The two retrieval channels are combined through a fixed linear (convex) combination, supplemented by an exact-keyword boost term. Denoting the dense similarity score as S_{dense} and the sparse similarity score as S_{sparse} , the fused score S_{hybrid} for a candidate FAQ entry is computed as:

$$S_{\text{hybrid}} = (0.65 \times S_{\text{dense}}) + (0.35 \times S_{\text{sparse}}) + \text{boost_exact}$$

where boost_exact is a small additive term applied when the query contains an exact keyword or phrase match against the candidate FAQ. The 0.65/0.35 weighting reflects a design decision, specified in the source project, to weight semantic similarity more heavily than lexical similarity while retaining lexical matching as a corrective signal. The fused ranking is then used both to select the top response and to construct a ranked list of alternative suggestions when the leading candidate's confidence falls below the system's acceptance threshold.

7.7 Sentiment-Driven Escalation

VADER sentiment scores are evaluated in real time against each customer turn. When sustained negative sentiment or repeated unresolved queries are detected within a session, the system generates an escalation record using a TICK-##### identifier scheme. Each ticket captures the issue category, a natural-language description, the customer's contact information, and an assigned priority level, enabling human agents to triage escalations without needing to replay the full conversation transcript.

7.8 Persistence and Audit Logging

All interaction data is persisted in a relational SQLite schema spanning conversations, messages, FAQ entries, interaction logs, feedback records, and tickets. Each logged interaction captures the customer query, the generated response, the classified intent, the retrieval confidence score, the sentiment label, end-to-end latency, and resolution status — a schema designed explicitly to support downstream analytics and to provide an auditable record for quality assurance.

7.9 Automated Testing

The supplied project repository specifies automated testing using Pytest in conjunction with the FastAPI TestClient, invoked via the command `pytest tests/ -v`. This testing strategy targets endpoint-level integration behaviour — verifying request validation, response schemas, and the

correctness of the retrieval and escalation logic under representative inputs — rather than relying solely on manual exploratory testing.

Chapter 8: Expected Outcomes and Evaluation Considerations

8.1 Expected Outcomes

- A functional, stateful customer-support chatbot capable of maintaining context across multi-turn conversations.
- Improved FAQ-retrieval robustness achieved through the combination of dense semantic and sparse lexical matching.
- Demonstrably better handling of paraphrased customer queries relative to a purely lexical baseline, attributable to dense sentence-embedding retrieval.
- Real-time sentiment awareness enabling the automated escalation of frustrated or unresolved interactions.
- A persistent, auditable record of support interactions and customer feedback suitable for retrospective analysis.
- Administrative visibility through key-performance-indicator dashboards, searchable logs, ticket queues, and CSV export.
- Dynamic FAQ knowledge-base management with on-demand re-indexing of the retrieval corpus.
- An NLP playground exposing intermediate model outputs to support explainability and debugging.
- A structured, documented REST API suitable for future integration with adjacent systems.

8.2 Evaluation Considerations

A rigorous evaluation of the hybrid retriever described in Chapter 7 would, in a production or research setting, report standard information-retrieval metrics — for example, top-k accuracy, Mean Reciprocal Rank (MRR), and Normalised Discounted Cumulative Gain (nDCG) — computed against a held-out set of customer queries with ground-truth FAQ labels, ideally benchmarked against the dense-only and sparse-only baselines in isolation to quantify the marginal contribution of fusion. The project materials supplied for this report do not include such benchmark results; accordingly, no quantitative performance figures are asserted here, and this is noted as a direction for future validation in Chapter 10 rather than presented as an established result.

Chapter 9: Limitations and Threats to Validity

A critical evaluation of the architecture described in this report surfaces several limitations that would need to be addressed before production deployment.

9.1 Fixed Fusion Weighting

The 0.65/0.35 dense–sparse fusion weighting is statically configured rather than learned or adapted per query. A learned fusion function — for instance, a lightweight logistic-regression re-ranker trained on labelled query–FAQ relevance pairs — would likely generalise better across query types than a single global weighting.

9.2 Retrieval Latency and Scalability

Dense retrieval requires an embedding-model inference pass per query and, without an approximate-nearest-neighbour index (e.g., FAISS or HNSW), a linear scan of cosine similarities against the full FAQ corpus. This is tractable for small-to-moderate FAQ sets but would require an indexed vector store to scale to large knowledge bases without degrading response latency.

9.3 Single-Node Persistence

SQLite is well suited to prototyping and moderate write volumes but is not designed for high-concurrency, multi-writer production workloads; a horizontally scalable deployment would require migration to a server-based relational database or an appropriately sharded store.

9.4 Lexicon-Based Sentiment Analysis

VADER is a lexicon- and rule-based sentiment model tuned primarily for social-media-style English text. It may misclassify sentiment expressed through sarcasm, domain-specific terminology, or languages other than English, which bears directly on the reliability of the escalation trigger.

9.5 Absence of Reported Benchmarks

As noted in Section 8.2, no quantitative accuracy, latency, or user-satisfaction benchmarks were included in the source materials for this report. Claims of “improved” retrieval performance should therefore be read as an architectural expectation grounded in the retrieval literature, not as an empirically validated result for this specific implementation.

Chapter 10: Future Work

- Replace the fixed dense–sparse fusion weighting with a learned re-ranking layer trained on logged interaction outcomes and explicit user feedback.
- Introduce an approximate-nearest-neighbour vector index to support sub-linear retrieval latency as the FAQ corpus scales.
- Migrate persistence from SQLite to a horizontally scalable relational or document store for multi-tenant, high-concurrency deployment.
- Conduct a controlled evaluation comparing dense-only, sparse-only, and hybrid retrieval using standard IR metrics (top-k accuracy, MRR, nDCG) against a labelled query set.
- Extend sentiment analysis with a fine-tuned transformer-based classifier to improve robustness to sarcasm and domain-specific phrasing.
- Integrate the platform with a live enterprise CRM and courier-tracking API to close the loop on order-related support scenarios, subject to the organisation's data-governance requirements.
- Add role-based access control and encryption at rest for the persistence layer ahead of any production deployment handling personally identifiable customer data.

Chapter 11: Certificate of Completion and Communication Proof

The certificate reproduced in the preliminary pages of this report (see page iv) states that Ayush Dixit completed a three-month, AICTE & ICAC approved internship programme at Codec Technologies Pvt. Ltd. between 28 May 2026 and 29 August 2026, in the role of Artificial Intelligence Intern. The certificate is countersigned by Dr. Anurag Shrivastava, Program Manager, and bears the identifier AICTE ID – CORPORATE6759d549ce59e1733940553.

No separate written communication record (for example, offer or completion correspondence) was included in the source materials supplied for this report. Consistent with academic-integrity requirements, no such record has been fabricated. Should the university's submission guidelines require additional communication proof beyond the certificate, that material should be appended by the candidate prior to final submission.

Bibliography / References

- [1] Bird, S., Klein, E., & Loper, E. Natural Language Processing with Python. NLTK Project official documentation.
- [2] Reimers, N., & Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Sentence-Transformers project documentation.
- [3] Pedregosa, F., et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research — TF-IDF and cosine-similarity reference documentation.
- [4] FastAPI Official Documentation — Python web-framework reference for building REST APIs.
- [5] Python Software Foundation. The Python Language Reference and Standard Library Documentation.
- [6] SQLite Consortium. SQLite Documentation — embedded relational database engine.
- [7] Wolf, T., et al. Transformers: State-of-the-Art Natural Language Processing. Hugging Face documentation and model hub.
- [8] Hutto, C.J., & Gilbert, E. VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. Association for the Advancement of Artificial Intelligence (AAAI).
- [9] Karpukhin, V., et al. Dense Passage Retrieval for Open-Domain Question Answering. Empirical Methods in Natural Language Processing (EMNLP) — background reference on dense retrieval.
- [10] Robertson, S., & Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval — background reference on sparse lexical retrieval.

Note: The project-specific technical facts presented in this report are derived from the supplied project README / repository description and the uploaded internship-completion certificate. Where the source materials did not specify a detail — including quantitative benchmarks, organisational history, or communication records — no such detail has been invented; the corresponding limitation is instead noted explicitly in Chapter 9.